实物交互光路系统复现说明

本说明对应本目录的 version14 PYNQ 与网页工程。复现顺序为：准备板卡与物块，启动 PYNQ 系统，准备电脑工程，复制板端运行文件，完成首次配置与本次开机准入，启动板端程序和电脑网页，操作实物完成关卡。

1 系统结构与运行文件

摄像头以两 lane、1280×720、RAW10/BGGR 方式采集。PL 承担 MIPI 视频接收、像素处理、帧搬运和 HDMI 输出；PYNQ Python 管理 Overlay、相机、GPIO、MMIO 与帧缓冲，并调用 C 库处理图像。C 核心完成彩色物块检测、姿态稳定、有效场景提交和光路求解。板端通过 UART v3 二进制协议发送姿态，电脑后台调用 C 游戏会话处理关卡、计时和结算，浏览器使用 Three.js 绘制模型与光路。

板端 HDMI 显示相机原图及板端实景诊断光路，诊断光源位于相机像素坐标 (32,500)，初始方向向右。网页中的光源位置与初始方向由所选关卡配置确定，游戏结果由电脑端 C 会话生成。

board/runtime/ 为直接部署的板端运行目录。src/project/ 为可直接使用的工程源码目录。sim/ 和 data/test_data/ 分别保存离线执行脚本与测试图片。

板端运行目录内的 libbeam_pynq.so、system_wrapper.bit、system_wrapper.hwh 和 hardware.json 位于 output/build/game_app/pynq_beam_hardware_5/。部署时整体保留该目录及其相对路径。

2 准备硬件与运行环境

2.1 设备和软件

项目	配置
开发板	AX7Z020B，Zynq-7020，SD 启动。
板端系统	AX7Z020B 配套 PYNQ 镜像；PYNQ 2.6.2、Python 3.6.5、ARMv7 Linux，GLIBC 2.27；内核 5.4.0-xilinx-v2020.1。
图像输入	工程配套 OV5640 MIPI 摄像头，两 lane，1280×720。
图像输出	HDMI 显示器，按 1280×720、16:9 显示。
连接线	摄像头排线、HDMI 线、板载 USB-UART 连接线、网线及板卡电源。
电脑环境	Windows、PowerShell、Python 3.12、OpenSSH 客户端及支持 WebGL 的浏览器。

项目	配置
源码工具	Visual Studio 2022 Professional，安装 C++ 桌面开发组件；源码构建使用 Vitis 2023.1 的 ARM Linux 交叉编译器，主机前端检查使用 Node.js。

将配套 PYNQ 系统镜像写入 microSD 卡，插入板卡并设置为 SD 启动。镜像写入完成后，启动文件与根文件系统由该 PYNQ 镜像提供。已有上述系统的板卡直接使用现有系统。

2.2 制作和摆放交互物块

本工程使用 restored_markers 检测入口，实物外观按下表制作。每个物块的主体、端部标记和轮廓均位于摄像头视野内。

物块	外观和标记
反射镜 Mirror	黄色主体，两端黑色圆点。
传送门 Portal A	蓝色主体，白色实心点与白色圆环分别位于两端。
传送门 Portal B	橙色主体，白色实心点与白色圆环分别位于两端。
目标 Target	绿色圆形主体。
障碍物 WALL	黑色主体，两端白色条形标记。

固定摄像头，使镜头朝向平整的操作区域，五类物块均能完整进入 1280×720 画面。固定光源由软件生成，场地中摆放反射镜、两个传送门、目标和障碍物。

2.3 连接和登录

在板卡断电时接好摄像头排线、HDMI、USB-UART 和网线。板卡通过网线直连电脑。将电脑该有线网口的 IPv4 地址设为 192.168.2.1，子网掩码设为 255.255.255.0；板卡使用配套镜像的直连地址 192.168.2.99。接通电源后，在 PowerShell 登录：

```
ssh xilinx@192.168.2.99
```

输入板卡的登录密码，进入 SSH 终端后执行 exit 返回电脑。之后的板端命令均在 SSH 终端执行，USB-UART 用于游戏业务数据。

3 准备电脑工程并安装依赖

在提交目录 pynq_web_v14_20261102/ 的根目录打开 PowerShell。以下命令将工程源码复制到提交目录旁的 beam_project/，后续电脑程序均从该目录启动。首次执行时，beam_project/ 为新目录。

```
$Package = (Get-Location).Path
$Project = Join-Path (Split-Path $Package -Parent) 'beam_project'
Set-ExecutionPolicy -Scope Process Bypass
Copy-Item -LiteralPath (Join-Path $Package 'src\project') `
  -Destination $Project -Recurse
Set-Location $Project
python -m venv .venv
$Py = Join-Path $Project '.venv\Scripts\python.exe'
& $Py -m pip install -r "$Package\build\requirements_host_locked.txt"
```

复制后，网页后台位于 pc_vision_1_/pc_vision/，共享 C 核心和配置位于 pc_vision/ 与 color_test/software/vision_ps_baremetal/。主配置为 pc_vision/game_config.json。网页资源及 Three.js 依赖已随工程保存，启动入口为工程根目录的 start_web.ps1。

执行电脑端检查，输出 C_GAME_READY 后继续：

```
.\start_web.ps1 -Check -PythonPath $Py
```

该命令准备并启动电脑端 C 游戏核心，完成检查后退出。C 可执行文件按源码和配置摘要存放在 pc_vision/output/board_native/；已有匹配构建时直接复用。

4 部署板端运行目录

在上述 PowerShell 终端继续执行，将 board/runtime/ 的内容复制到板卡固定目录，再进入 SSH：

```
$Board = 'xilinx@192.168.2.99'
$Runtime = Join-Path $Package 'board\runtime'
ssh $Board 'mkdir -p /home/xilinx/jupyter_notebooks/beam_v14'
scp -r "$Runtime/." "${Board}:/home/xilinx/jupyter_notebooks/beam_v14/"
ssh $Board
```

板端目标目录包含 beam_v14.py、两个 .pyz 工具、pynq_vision/ 和 output/。在 SSH 终端进入该目录，检查文件和当前系统：

```
cd /home/xilinx/jupyter_notebooks/beam_v14
python3 -B beam_v14.py --inspect
sudo python3 -B beam_v14.py --check
```

--inspect 核对运行文件与硬件清单的摘要；--check 输出 Python、内核、本次 boot_id、控制台、I2C 路由和设备占用信息。运行设备使用 /dev/ttyPS0 与 /dev/i2c-0。

4.1 首次配置业务串口

在 SSH 终端运行部署工具：

```
sudo python3 -B beam_v14_deploy_v2.pyz --audit
sudo python3 -B beam_v14_deploy_v2.pyz --apply-config
```

配置工具核对配套镜像的启动脚本和网口配置，保存实际原文件到 /root/beam_v14_setup_backups/，将内核控制台设为 tty0，并通过启动参数屏蔽 ttyPS0 的串口登录服务。直连网口使用 192.168.2.99；镜像已有静态别名时保留该配置。工具输出 configuration，记录实际变更和备份位置。

首次配置完成后重启板卡：

```
sudo reboot
```

等待系统重新启动，在电脑执行 ssh xilinx@192.168.2.99，再进入 beam_v14/。完成首次配置后的日常启动从第 5 节执行。

5 准入并启动板端程序

本次开机保持板端程序、Notebook 和串口终端均未占用相机、显示及业务 UART。在 SSH 终端执行：

```
cd /home/xilinx/jupyter_notebooks/beam_v14
sudo -v
sudo -n true
cat /proc/sys/kernel/random/boot_id
sudo python3 -B beam_v14.py --check
sudo python3 -B beam_v14_ready_v2.pyz --qualify
```

准入工具核对当前开机身份、设备所有者和 PS 隔离状态，加载配套 Overlay，检查 CSI 版本、时钟及采集和显示 DMA 状态。输出 ADMITTED_FROM_ACTUAL_HARDWARE 后，在 /run/beam_pynq/ 写入本次开机的受保护准入与证据文件。

随后启动连续采集、识别和 HDMI 显示：

```
sudo python3 -u -B beam_v14.py --run
```

保持该 SSH 终端运行。在操作区放置并短暂保持物块静止，HDMI 依次显示相机画面、物体轮廓、姿态与完整光路。移动或旋转物块后，新观测经过稳定确认，再提交新的完整画面。显示暂存画面保留原采集时间，状态栏显示 STALE 和帧龄。

同一次开机正常退出后再次启动，直接执行 --run。系统重启后，先执行一次 --qualify，再执行 --run。准入命令以 ADMITTED_FROM_ACTUAL_HARDWARE 作为进入运行的结果；输出 STOPPED 时保持该轮输出和现场状态，结束本轮启动。

6 启动电脑网页并操作游戏

6.1 启动串口后台

保留板端 SSH 会话，在电脑另开 PowerShell，进入第 3 节复制的 beam_project/。通过设备列表确定板载 USB-UART 的 COM 端口：

```
Get-CimInstance Win32_SerialPort | Select-Object DeviceID, Name
```

关闭占用该 COM 端口的串口终端。以下以 COM9 为例；将 SerialPort 的值填写为设备列表中对应的端口，执行：

```
$Py = (Resolve-Path '.\.venv\Scripts\python.exe').Path
Set-ExecutionPolicy -Scope Process Bypass
.\start_web.ps1 -SerialPort COM9 -BaudRate 115200 `
  -Port 8765 -PythonPath $Py
```

后台输出 Board port、UART trace 和 Web UI。用浏览器打开 http://127.0.0.1:8765/。网页资源、HTTP 服务、串口接收和 C 游戏会话均在电脑运行；板卡网口用于 SSH 管理，游戏数据经 USB-UART 传输。

6.2 选关、开始与结算

1. 在网页选择关卡，后台向板端发送带关卡、代次和 nonce 的 LEVEL 消息。将该关要求的物块放入摄像头视野，保持端部标记完整可见，等待关卡就绪和开始按钮解锁。
2. 点击“开始计时”。START 经板端确认后，本关进入 180 秒计时。准备阶段保持 03:00。
3. 平移或原地旋转反射镜、传送门及其他物块。摄像头测得的有效姿态经串口传入电脑 C 会话，网页随已提交场景更新模型和光路；反射镜改变反射方向，传送门改变传播路径，障碍物终止光线，目标接收命中。
4. 使光线命中 Target，网页进入关卡结算。用时小于 60、120、180 秒分别为 3、2、1 星；达到 180 秒为 0 星。
5. 点击“下一关”继续，或点击“返回选关”重新选择。运行中可使用“跳过”进入下一关。鼠标拖动与滚轮分别控制观察视角和缩放，实物位置由相机输入。

网页显示“板端已连接”时，后台已接入业务串口。关卡就绪、开始计时和结算分别由对应控制确认和 C 会话状态驱动。

6.3 结束运行

先在电脑后台终端按 Ctrl+C，等待 HTTP、串口和 C 会话退出。再在板端 SSH 终端按 Ctrl+C，等待采集与显示 DMA 停机、缓冲释放和统计输出，返回提示符后结束会话。板端统计中的 can_exit 为 true 时，资源已完成正常退出；出现 BEAM_OWNER_RETAINED 时保持所有者进程。

需要关闭板卡时，在板端正常退出后执行 sudo poweroff，等待系统关机后断电。

7 从源码构建与离线回放

第 3 至第 6 节使用提交目录内的配套运行制品。源码构建使用同一份工程源码、同一 C 核心和同一 JSON，执行入口如下。

7.1 构建 C 原生库

将第 2 节使用的镜像保存为 ax7z020b-2.6.0.img，与提交目录和 beam_project/ 放在同一级目录。在 beam_project/ 根目录打开 PowerShell，将 Vitis 的 arm-linux-gnueabihf-gcc.exe 所在目录加入当前终端 PATH，然后设置路径：

```
$Package = (Resolve-Path '..\pynq_web_v14_20261102').Path
$Project = (Get-Location).Path
$Py = (Resolve-Path '.\.venv\Scripts\python.exe').Path
$ArmCc = (Get-Command arm-linux-gnueabihf-gcc.exe).Source
$Image = (Resolve-Path '..\ax7z020b-2.6.0.img').Path
$Tools = Join-Path $Project 'color_test\software\vision_ps_baremetal\tools'
$Libs = Join-Path $Project 'output\build\game_app\pynq_target_libs_1'
```

安装镜像读取依赖，从同一系统镜像中提取 ARM 链接库，再构建 C 原生库：

```
& $Py -m pip install dissect.extfs==3.15 dissect.util==3.24 `
  dissect.cstruct==4.7
& $Py -B "$Tools\prepare_beam_target_libs.py" `
  --image $Image --output $Libs
& $Py -B "$Package\build\build_c.py" `
  --project $Project --output "$Project\output\rebuild_c_1" `
  --arm-cc $ArmCc --target-libs $Libs
```

构建脚本从 pc_vision/game_config.json 生成 PYNQ 配置，编译 Cortex-A9 hard-float ARM 共享库和主机测试库，检查 ELF ABI 与 GLIBC 版本需求，并执行 C 运行层主机测试。输出位于 output/rebuild_c_1/，包含 libbeam_pynq.so、beam_pynq_host.dll、配置、构建日志和 build.json。目标系统库目录中的 GLIBC 2.27 文件作为交叉链接输入使用。

本复现的硬件输入取自配套 system_wrapper.bit 与 system_wrapper.hwh，二者由同一 hardware.json 绑定。Python 通过 Overlay 装载该输入，C 原生库通过平台回调使用现有采集和显示链路。

7.2 执行主机检查

在同一 PowerShell 终端执行：

```
& $Py -B "$Package\sim\run_checks.py" `
  --project $Project --output "$Project\output\host_checks_1" `
  --host-library "$Project\output\rebuild_c_1\beam_pynq_host.dll"
```

脚本依次执行 PYNQ 运行层与硬件所有者模型、准入与部署工具、UART v3、网页后台、前端测试和文件检查。各项退出码、日志及汇总保存在 output/host_checks_1/。

7.3 回放原始测试图片

在同一 PowerShell 终端执行：

```
& $Py -B "$Package\sim\replay_test_data.py" `
  --project $Project --index "$Package\data\test_data\index.tsv" `
  --output "$Project\output\photo_replay_1"
```

脚本按索引读取 21 张图片：01.png 至 20.png，以及 42.png，解码为原分辨率 BGR888，编译并调用交付 C 检测器和光路核心。输出目录包含逐图日志、物体中心与轴向、有效模型、完整光段、原图与计算结果对照图，以及 summary.tsv 和 results.json。该过程复现静态图片的检测与光路计算；实时相机、姿态连续确认和网页游戏按第 5、第 6 节运行。

以上构建和回放输出均使用新目录，原始源码、配套运行目录和已有结果保留在各自路径。